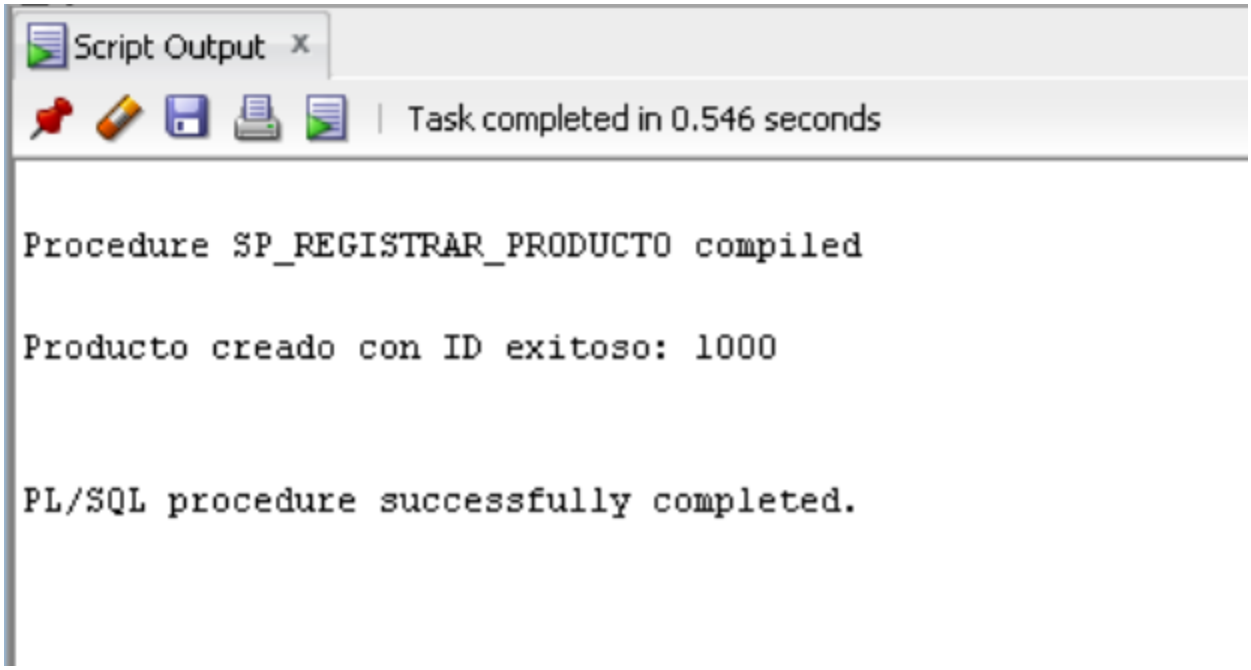
Evidencias grupo 8

Integrantes:
Gabriella Castro
Lina Patiño
Julián Rodríguez

Fecha de entrega
3 de Octubre de 2026

1.1
SCRIPT PARA PROBAR PROCEDIMIENTO

```
SET SERVEROUTPUT ON;
DECLARE
    v_id_prod NUMBER;
BEGIN
    SP_REGISTRAR_PRODUCTO(
        p_codigo       => 'PRU-100',
        p_nombre       => 'Alicate de prueba',
        p_id_categoria => 1,
        p_precio       => 50000,
        p_costo        => 30000,
        p_stock_minimo => 10,
        p_id_producto  => v_id_prod
    );
    DBMS_OUTPUT.PUT_LINE('Producto creado con ID exitoso: ' ||
v_id_prod);
END;
/
```



1.2
SCRIPT PARA PROBAR PROCEDIMIENTO

```
SET SERVEROUTPUT ON;

DECLARE
    v_saldo_res NUMBER;
BEGIN
    SP_AJUSTAR_STOCK(
        p_id_producto => 1001,
        p_id_bodega   => 1,
        p_tipo        => 'ENTRADA',
        p_cantidad    => 20,
        p_observacion => 'Prueba de entrada - Evidencia Taller',
        p_saldo_final => v_saldo_res
    );
    DBMS_OUTPUT.PUT_LINE('--> ENTRADA EXITOSA. Saldo final en bodega:
' || v_saldo_res);

    SP_AJUSTAR_STOCK(
        p_id_producto => 1001,
        p_id_bodega   => 1,
        p_tipo        => 'SALIDA',
        p_cantidad    => 5,
        p_observacion => 'Prueba de salida - Evidencia Taller',
        p_saldo_final => v_saldo_res
```

```
    );
    DBMS_OUTPUT.PUT_LINE('--> SALIDA EXITOSA. Saldo final en bodega:
' || v_saldo_res);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error atrapado en prueba: ' ||
SQLERRM);
END;
/

SELECT id_movimiento, id_producto, id_bodega, tipo, cantidad,
observacion
FROM MOVIMIENTO_INVENTARIO
ORDER BY id_movimiento DESC
FETCH FIRST 2 ROWS ONLY;
```

```
Procedure SP_AJUSTAR_STOCK compiled
```

```
--> ENTRADA EXITOSA. Saldo final en bodega: 140
--> SALIDA EXITOSA. Saldo final en bodega: 135


PL/SQL procedure successfully completed.


ID_MOVIMIENTO ID_PRODUCTO  ID_BODEGA TIPO         CANTIDAD OBSERVACION
------------- ----------- ---------- ---------- ---------- --------------------------------------------------------------------------------
           22        1001          1 SALIDA              5 Prueba de salida - Evidencia Taller
           21        1001          1 ENTRADA            20 Prueba de entrada - Evidencia Taller
```

1.3

Primero se corrió el punto 2.2 (necesario para que SP_CREAR_PEDIDO valide el cupo de crédito disponible mediante FN_CUPO_DISPONIBLE).

```
Procedure SP_CREAR_PEDIDO compiled
```

SCRIPT PARA PROBAR PROCEDIMIENTO

```
SET SERVEROUTPUT ON;

DECLARE
    V_id_ped NUMBER;
BEGIN
```

```
    -- Prueba exitosa con el cliente 501 (Constructora Andes SAS -
activo y con cupo)
    SP_CREAR_PEDIDO(
        p_id_cliente => 501,
        p_id_pedido  => V_id_ped
    );
    DBMS_OUTPUT.PUT_LINE('--> PEDIDO BORRADOR CREADO CON ÉXITO. ID de
Pedido: ' || V_id_ped);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error atrapado en prueba: ' ||
SQLERRM);
END;
/

-- Verificación en la base de datos de la cabecera del pedido creada
en estado BORRADOR:
SELECT id_pedido, id_cliente, fecha_pedido, estado, total
FROM PEDIDO
WHERE estado = 'BORRADOR'
ORDER BY id_pedido DESC
FETCH FIRST 1 ROWS ONLY;
```

```
--> PEDIDO BORRADOR CREADO CON ÉXITO. ID de Pedido: 8


PL/SQL procedure successfully completed.


 ID_PEDIDO ID_CLIENTE FECHA_PED ESTADO               TOTAL
---------- ---------- --------- --------------- ----------
         8        501 03-OCT-26 BORRADOR                 0
```

2.1

SCRIPT PARA PROBAR FUNCIÓN

```
SET SERVEROUTPUT ON;

DECLARE
```

```
    V_stock_bodega NUMBER;
    V_stock_total  NUMBER;
    V_stock_inix   NUMBER;
BEGIN
    -- 1. Probar con bodega específica (Ej: Producto 1001 en Bodega
1)
    V_stock_bodega := FN_STOCK_DISPONIBLE(1001, 1);
    DBMS_OUTPUT.PUT_LINE('--> Stock prod 1001 en Bodega 1: ' ||
V_stock_bodega);

    -- 2. Probar consolidado (Bodega nula, ej: Producto 1001 en todas
las bodegas)
    V_stock_total := FN_STOCK_DISPONIBLE(1001, NULL);
    DBMS_OUTPUT.PUT_LINE('--> Stock consolidado prod 1001: ' ||
V_stock_total);

    -- 3. Probar con un producto inexistente (Ej: Producto 9999, debe
retornar 0 sin error)
    V_stock_inix := FN_STOCK_DISPONIBLE(9999, NULL);
    DBMS_OUTPUT.PUT_LINE('--> Stock de producto inexistente (9999): '
|| V_stock_inix);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error atrapado en prueba: ' ||
SQLERRM);
END;
/

-- Verificación directa mediante consulta SQL:
SELECT id_producto, codigo, nombre, FN_STOCK_DISPONIBLE(id_producto,
NULL) AS stock_consolidado
FROM PRODUCTO
WHERE id_producto IN (1001, 9999);
```

```
Function FN_STOCK_DISPONIBLE compiled
```

```
--> Stock prod 1001 en Bodega 1: 135
--> Stock consolidado prod 1001: 175
--> Stock de producto inexistente (9999): 0


PL/SQL procedure successfully completed.


ID_PRODUCTO CODIGO           NOMBRE                                                                                                STOCK_CONSOLIDADO
----------- ---------------- ----------------------------------------------------------------------------------------------------- ----------------
       1001 HM-0001          Martillo carpintero 16oz                                                                                           175
```

2.2

SCRIPT PARA PROBAR FUNCIÓN

```
SET SERVEROUTPUT ON;

DECLARE
    V_cupo_disp NUMBER;
BEGIN
    -- 1. Probar con un cliente activo y con cupo (Ej: Cliente 501 -
Constructora Andes SAS)
    V_cupo_disp := FN_CUPO_DISPONIBLE(501);
    DBMS_OUTPUT.PUT_LINE('--> Cupo disponible cliente 501: ' ||
V_cupo_disp);

    -- 2. Probar con un cliente inactivo (Ej: Cliente 506 -
Suministros del Caribe SA, debe retornar 0)
    V_cupo_disp := FN_CUPO_DISPONIBLE(506);
    DBMS_OUTPUT.PUT_LINE('--> Cupo disponible cliente inactivo 506: '
|| V_cupo_disp);

    -- 3. Probar con un cliente inexistente (Ej: Cliente 9999, debe
retornar 0)
    V_cupo_disp := FN_CUPO_DISPONIBLE(9999);
    DBMS_OUTPUT.PUT_LINE('--> Cupo disponible cliente inexistente
9999: ' || V_cupo_disp);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error atrapado en prueba: ' ||
SQLERRM);
END;
/

-- Verificación directa mediante consulta SQL para todos los
clientes:
SELECT id_cliente, nombre, cupo_credito, saldo_pendiente, activo,
FN_CUPO_DISPONIBLE(id_cliente) AS cupo_disponible
FROM CLIENTE
ORDER BY id_cliente;
```

```
Function FN_CUPO_DISPONIBLE compiled
```

```
PL/SQL procedure successfully completed.


ID_CLIENTE NOMBRE                                                                                                CUPO_CREDITO SALDO_PENDIENTE A CUPO_DISPONIBLE
---------- ----------------------------------------------------------------------------------------------------- ------------ --------------- - --------------
       501 Constructora Andes SAS                                                                                    8000000         1250000 S        6750000
       502 Electricos del Norte Ltda                                                                                 5000000         4700000 S         300000
       503 Ferreteria La Esquina                                                                                     2000000               0 S        2000000
       504 Obras Civiles del Valle                                                                                  12000000         3400000 S        8600000
       505 Mantenimientos Tecnicos                                                                                   1500000         1500000 S              0
       506 Suministros del Caribe SA                                                                                 3000000               0 N              0
       507 Distribuidora Caribe                                                                                      6000000          800000 S        5200000

7 rows selected.
```

2.3

SCRIPT PARA PROBAR FUNCIÓN

```
SET SERVEROUTPUT ON;

DECLARE
    V_desc NUMBER;
BEGIN
    -- 1. Probar con monto >= 2.000.000 y cliente con saldo pendiente
(Ej: Cliente 501, monto 2.500.000 -> base 8%)
    V_desc := FN_CALCULAR_DESCUENTO(501, 2500000);
    DBMS_OUTPUT.PUT_LINE('--> Descuento cliente 501 (monto 2.5M): '
|| V_desc);

    -- 2. Probar con monto entre 800k y 1.99M y cliente SIN saldo
pendiente (Ej: Cliente 503, saldo 0 -> base 5% + 2% adicional = 7%)
    V_desc := FN_CALCULAR_DESCUENTO(503, 1000000);
    DBMS_OUTPUT.PUT_LINE('--> Descuento cliente 503 sin saldo (monto
1M): ' || V_desc);

    -- 3. Probar con monto menor a 800.000 y cliente SIN saldo
pendiente (Ej: Cliente 503, monto 500.000 -> base 0% + 2% adicional =
2%)
    V_desc := FN_CALCULAR_DESCUENTO(503, 500000);
    DBMS_OUTPUT.PUT_LINE('--> Descuento cliente 503 sin saldo (monto
500k): ' || V_desc);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error atrapado en prueba: ' ||
SQLERRM);
```

```
END;
/

-- Verificación directa mediante consulta SQL para evaluar diferentes
montos y clientes:
SELECT id_cliente, nombre, saldo_pendiente,
       FN_CALCULAR_DESCUENTO(id_cliente, 2500000) AS desc_2_5m,
       FN_CALCULAR_DESCUENTO(id_cliente, 1000000) AS desc_1m
FROM CLIENTE
WHERE id_cliente IN (501, 503);
```

```
Function FN_CALCULAR_DESCUENTO compiled
```

```
--> Descuento cliente 501 (monto 2.5M): 200000
--> Descuento cliente 503 sin saldo (monto 1M): 70000
--> Descuento cliente 503 sin saldo (monto 500k): 10000


PL/SQL procedure successfully completed.


ID_CLIENTE NOMBRE                                                                                                SALDO_PENDIENTE  DESC_2_5M    DESC_1M
---------- ----------------------------------------------------------------------------------------------------- --------------- ---------- ----------
       501 Constructora Andes SAS                                                                                      1250000     200000      50000
       503 Ferreteria La Esquina                                                                                             0     250000      70000
```

2.4

	ID_CLIENTE	DOCUMENTO	NOMBRE	CIUDAD	CUPO_DISPONIBLE
1	504	900456789	Obras Civiles del Valle	Cali	8600000
2	501	900123456	Constructora Andes SAS	Bogota	6750000
3	507	900789012	Distribuidora Caribe	Barranquilla	5200000
4	503	900345678	Ferreteria La Esquina	Medellin	2000000
5	502	900234567	Electricos del Norte Ltda	Bogota	300000
6	505	900567890	Mantenimientos Tecnicos	Bogota	0

3.1
SCRIPT PARA PROBAR CURSOR EXPLÍCITO

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR c_bajo_stock IS
```

```
       SELECT p.codigo,
              p.nombre,
              NVL(SUM(e.cantidad), 0) AS stock_actual,
              p.stock_minimo,
              p.activo
       FROM   PRODUCTO p
              LEFT JOIN EXISTENCIA e ON e.id_producto = p.id_producto
       GROUP BY p.id_producto, p.codigo, p.nombre, p.stock_minimo,
p.activo
       HAVING NVL(SUM(e.cantidad), 0) < p.stock_minimo
       ORDER BY p.codigo;

   v_codigo  PRODUCTO.codigo%TYPE;
   v_nombre  PRODUCTO.nombre%TYPE;
   v_stock   NUMBER;
   v_minimo  PRODUCTO.stock_minimo%TYPE;
   v_activo  PRODUCTO.activo%TYPE;
   v_total   PLS_INTEGER;
BEGIN
   DBMS_OUTPUT.PUT_LINE('REPORTE: PRODUCTOS POR DEBAJO DEL STOCK
MINIMO');
   DBMS_OUTPUT.PUT_LINE(RPAD('CODIGO', 10) || RPAD('NOMBRE', 34) ||
                        LPAD('ACTUAL', 8) || LPAD('MINIMO', 8) ||
LPAD('FALTAN', 10));
   DBMS_OUTPUT.PUT_LINE(RPAD('-', 70, '-'));

   OPEN c_bajo_stock;
   LOOP
       FETCH c_bajo_stock INTO v_codigo, v_nombre, v_stock, v_minimo,
v_activo;
       EXIT WHEN c_bajo_stock%NOTFOUND;

       DBMS_OUTPUT.PUT_LINE(RPAD(v_codigo, 10) || RPAD(v_nombre, 34)
||
                            LPAD(v_stock, 8) || LPAD(v_minimo, 8) ||
                            LPAD(v_minimo - v_stock, 10) ||
                            CASE WHEN v_activo = 'N' THEN '
(INACTIVO)' END);
   END LOOP;
   v_total := c_bajo_stock%ROWCOUNT;   -- se lee antes del CLOSE
   CLOSE c_bajo_stock;

   DBMS_OUTPUT.PUT_LINE(RPAD('-', 70, '-'));
   DBMS_OUTPUT.PUT_LINE('Total de productos bajo el stock minimo: '
|| v_total);
```

```
EXCEPTION
   WHEN OTHERS THEN
       IF c_bajo_stock%ISOPEN THEN
           CLOSE c_bajo_stock;
       END IF;
       RAISE;
END;
/
```

```
Procedimiento PL/SQL terminado correctamente.
REPORTE: PRODUCTOS POR DEBAJO DEL STOCK MINIMO
CODIGO    NOMBRE                              ACTUAL  MINIMO    FALTAN
-------------------------------------------------------------------
FG-0002   Cinta metrica 8 metros                   9      25        16
HM-0004   Alicate universal 8 pulgadas             0      20        20  (INACTIVO)
ME-0001   Cable THHN 12 AWG metro                470     500        30
PRU-100   Alicate de prueba                       0      10        10
-------------------------------------------------------------------
Total de productos bajo el stock minimo: 4

Procedimiento PL/SQL terminado correctamente.
```

3.2

SCRIPT PARA PROBAR CURSOR PARAMETRIZADO

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR c_pedidos (p_id_cliente NUMBER, p_desde DATE, p_hasta DATE)
IS
       SELECT pe.id_pedido,
              pe.fecha_pedido,
              pe.total,
              COUNT(d.linea) AS num_lineas
       FROM   PEDIDO pe
              LEFT JOIN DETALLE_PEDIDO d ON d.id_pedido =
pe.id_pedido
       WHERE  pe.id_cliente   = p_id_cliente
       AND    pe.estado       = 'CONFIRMADO'
       AND    pe.fecha_pedido >= p_desde
       AND    pe.fecha_pedido <  p_hasta + 1
       GROUP BY pe.id_pedido, pe.fecha_pedido, pe.total
       ORDER BY pe.fecha_pedido, pe.id_pedido;

   v_cuenta  PLS_INTEGER := 0;
   v_suma    NUMBER      := 0;
```

```
BEGIN
   DBMS_OUTPUT.PUT_LINE('PEDIDOS CONFIRMADOS DEL CLIENTE 501 EN
2026');
   DBMS_OUTPUT.PUT_LINE(RPAD('PEDIDO', 10) || RPAD('FECHA', 14) ||
                        LPAD('TOTAL', 14) || LPAD('LINEAS', 8));
   DBMS_OUTPUT.PUT_LINE(RPAD('-', 46, '-'));

   FOR r IN c_pedidos(501, DATE '2026-01-01', DATE '2026-12-31') LOOP
       v_cuenta := v_cuenta + 1;
       v_suma   := v_suma + r.total;
       DBMS_OUTPUT.PUT_LINE(RPAD(r.id_pedido, 10) ||
                            RPAD(TO_CHAR(r.fecha_pedido,
'YYYY-MM-DD'), 14) ||
                            LPAD(TO_CHAR(r.total,
'FM999G999G999G990'), 14) ||
                            LPAD(r.num_lineas, 8));
   END LOOP;

   IF v_cuenta = 0 THEN
       DBMS_OUTPUT.PUT_LINE('El cliente no tiene pedidos confirmados
en ese rango.');
   ELSE
       DBMS_OUTPUT.PUT_LINE(RPAD('-', 46, '-'));
       DBMS_OUTPUT.PUT_LINE('Pedidos: ' || v_cuenta ||
                            '   Suma de totales: ' || TO_CHAR(v_suma,
'FM999G999G999G990'));
   END IF;
END;
/
```

```
PEDIDOS CONFIRMADOS DEL CLIENTE 501 EN 2026
PEDIDO    FECHA                 TOTAL  LINEAS
----------------------------------------------
1         2026-06-10        1.286.500       3
2         2026-07-02          641.000       2
----------------------------------------------
Pedidos: 2   Suma de totales: 1.927.500

Procedimiento PL/SQL terminado correctamente.
```

3.3

SCRIPT PARA PROBAR CURSOR FOR UPDATE

```
SET SERVEROUTPUT ON;
```

```
DECLARE
   v_id_categoria  NUMBER := 1;      -- 1 = Herramienta manual
(margen 25%)
   v_pct           NUMBER := -22;    -- variacion porcentual a
aplicar

   CURSOR c_prod (p_id_categoria NUMBER) IS
       SELECT p.id_producto,
              p.codigo,
              p.precio_unitario,
              p.costo_unitario,
              c.margen_minimo
       FROM   PRODUCTO p
              JOIN CATEGORIA c ON c.id_categoria = p.id_categoria
       WHERE  p.id_categoria = p_id_categoria
       AND    p.activo = 'S'
       ORDER BY p.id_producto
       FOR UPDATE OF p.precio_unitario WAIT 5;

   e_recurso_ocupado EXCEPTION;
   PRAGMA EXCEPTION_INIT(e_recurso_ocupado, -30006);

   v_nuevo        NUMBER;
   v_precio_min   NUMBER;
   v_actualizados PLS_INTEGER := 0;
   v_omitidos     PLS_INTEGER := 0;
BEGIN
   DBMS_OUTPUT.PUT_LINE('AJUSTE DE PRECIOS CATEGORIA ' ||
v_id_categoria ||
                        ' (' || v_pct || '%)');

   FOR r IN c_prod(v_id_categoria) LOOP
       v_nuevo      := ROUND(r.precio_unitario * (1 + v_pct / 100),
2);
       v_precio_min := r.costo_unitario * (1 + r.margen_minimo /
100);

       IF v_nuevo < v_precio_min THEN
           v_omitidos := v_omitidos + 1;
           DBMS_OUTPUT.PUT_LINE(RPAD(r.codigo, 10) || 'OMITIDO: ' ||
v_nuevo ||
                                ' quedaria bajo el minimo ' ||
v_precio_min);
       ELSE
           UPDATE PRODUCTO
```

```
            SET    precio_unitario = v_nuevo
            WHERE CURRENT OF c_prod;
            v_actualizados := v_actualizados + 1;
            DBMS_OUTPUT.PUT_LINE(RPAD(r.codigo, 10) || 'ACTUALIZADO: '
||
                                 r.precio_unitario || ' -> ' ||
v_nuevo);
        END IF;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Actualizados: ' || v_actualizados ||
                         '   Omitidos por margen: ' || v_omitidos ||
                         '   (los inactivos no se leen)');

    ROLLBACK;   -- deja los datos de prueba como estaban; en uso real:
COMMIT
EXCEPTION
    WHEN e_recurso_ocupado THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Filas bloqueadas por otra sesion
(ORA-30006). Reintente mas tarde.');
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
/
```

```
AJUSTE DE PRECIOS CATEGORIA 1 (-22%)
PRU-100    ACTUALIZADO: 50000 -> 39000
HM-0001    ACTUALIZADO: 45000 -> 35100
HM-0002    ACTUALIZADO: 62000 -> 48360
HM-0003    OMITIDO: 29640 quedaria bajo el minimo 30000
Actualizados: 3   Omitidos por margen: 1   (los inactivos no se leen)

Procedimiento PL/SQL terminado correctamente.
```

3.4

SCRIPT PARA PROBAR PROCEDIMIENTO CON CURSOR DE REFERENCIA

```
CREATE OR REPLACE PROCEDURE SP_LISTAR_MOVIMIENTOS (
    p_id_producto IN  NUMBER,
    p_cursor      OUT SYS_REFCURSOR
) IS
BEGIN
    OPEN p_cursor FOR
        SELECT id_movimiento,
```

```
                id_bodega,
                tipo,
                cantidad,
                fecha_mov,
                id_pedido,
                observacion
        FROM   MOVIMIENTO_INVENTARIO
        WHERE  id_producto = p_id_producto
        ORDER BY fecha_mov, id_movimiento;
END SP_LISTAR_MOVIMIENTOS;
/

-- Bloque consumidor (producto 1001):
DECLARE
    v_cursor   SYS_REFCURSOR;
    v_id_mov   MOVIMIENTO_INVENTARIO.id_movimiento%TYPE;
    v_bodega   MOVIMIENTO_INVENTARIO.id_bodega%TYPE;
    v_tipo     MOVIMIENTO_INVENTARIO.tipo%TYPE;
    v_cantidad MOVIMIENTO_INVENTARIO.cantidad%TYPE;
    v_fecha    MOVIMIENTO_INVENTARIO.fecha_mov%TYPE;
    v_pedido   MOVIMIENTO_INVENTARIO.id_pedido%TYPE;
    v_obs      MOVIMIENTO_INVENTARIO.observacion%TYPE;
    v_total    PLS_INTEGER := 0;
BEGIN
    SP_LISTAR_MOVIMIENTOS(1001, v_cursor);

    DBMS_OUTPUT.PUT_LINE('MOVIMIENTOS DEL PRODUCTO 1001');
    DBMS_OUTPUT.PUT_LINE(RPAD('MOV', 6) || RPAD('FECHA', 12) ||
RPAD('TIPO', 10) ||
                         LPAD('BODEGA', 8) || LPAD('CANT', 8) ||
LPAD('PEDIDO', 8) ||
                         '  OBSERVACION');
    LOOP
        FETCH v_cursor INTO v_id_mov, v_bodega, v_tipo, v_cantidad,
v_fecha, v_pedido, v_obs;
        EXIT WHEN v_cursor%NOTFOUND;
        v_total := v_total + 1;
        DBMS_OUTPUT.PUT_LINE(RPAD(v_id_mov, 6) ||
                             RPAD(TO_CHAR(v_fecha, 'YYYY-MM-DD'), 12)
||
                             RPAD(v_tipo, 10) || LPAD(v_bodega, 8) ||
                             LPAD(v_cantidad, 8) ||
LPAD(NVL(TO_CHAR(v_pedido), '-'), 8) ||
                             '  ' || v_obs);
    END LOOP;
```

```
    CLOSE v_cursor;   -- el consumidor es quien cierra el cursor

    DBMS_OUTPUT.PUT_LINE('Total de movimientos: ' || v_total);
EXCEPTION
    WHEN OTHERS THEN
        IF v_cursor%ISOPEN THEN
            CLOSE v_cursor;
        END IF;
        RAISE;
END;
/
```

```
Procedure SP_LISTAR_MOVIMIENTOS compilado

MOVIMIENTOS DEL PRODUCTO 1001
MOV   FECHA       TIPO        BODEGA    CANT  PEDIDO  OBSERVACION
1     2026-05-02  ENTRADA          1     150       -  Compra a proveedor
6     2026-06-10  SALIDA           1       5       1  Despacho de pedido
8     2026-06-20  ENTRADA          2      40       -  Traslado entre bodegas
20    2026-08-25  AJUSTE           1      25       -  Conteo fisico
21    2026-10-03  ENTRADA          1      20       -  Prueba de entrada - Evidencia Taller
22    2026-10-03  SALIDA           1       5       -  Prueba de salida - Evidencia Taller
Total de movimientos: 6

Procedimiento PL/SQL terminado correctamente.
```

3.5

SCRIPT PARA PROBAR BULK COLLECT Y FORALL

```
SET SERVEROUTPUT ON;

DECLARE
   TYPE t_ids    IS TABLE OF PRODUCTO.id_producto%TYPE;
   TYPE t_minimo IS TABLE OF PRODUCTO.stock_minimo%TYPE;

   v_id_categoria  NUMBER := 3;      -- 3 = Material electrico
   v_factor        NUMBER := 1.10;   -- aumento del 10%
   v_ids           t_ids;
   v_nuevos        t_minimo;
BEGIN
   SELECT id_producto,
          CEIL(stock_minimo * v_factor)
   BULK COLLECT INTO v_ids, v_nuevos
   FROM   PRODUCTO
   WHERE  id_categoria = v_id_categoria
   AND    activo = 'S'
   ORDER BY id_producto;
```

```
   DBMS_OUTPUT.PUT_LINE('Productos cargados en la coleccion: ' ||
v_ids.COUNT);

   IF v_ids.COUNT > 0 THEN
       FORALL i IN 1 .. v_ids.COUNT
           UPDATE PRODUCTO
           SET    stock_minimo = v_nuevos(i)
           WHERE  id_producto  = v_ids(i);

       FOR i IN 1 .. v_ids.COUNT LOOP
           DBMS_OUTPUT.PUT_LINE('Producto ' || v_ids(i) ||
                                ' -> nuevo stock minimo ' ||
v_nuevos(i) ||
                                ' (filas: ' || SQL%BULK_ROWCOUNT(i)
|| ')');
       END LOOP;
       DBMS_OUTPUT.PUT_LINE('Filas actualizadas en total: ' ||
SQL%ROWCOUNT);
   END IF;

   ROLLBACK;   -- deja los datos de prueba como estaban; en uso real:
COMMIT
EXCEPTION
   WHEN OTHERS THEN
       ROLLBACK;
       RAISE;
END;
/
```

```
Productos cargados en la coleccion: 3
Producto 1007 -> nuevo stock minimo 550 (filas: 1)
Producto 1008 -> nuevo stock minimo 44 (filas: 1)
Producto 1009 -> nuevo stock minimo 66 (filas: 1)
Filas actualizadas en total: 3

Procedimiento PL/SQL terminado correctamente.
```

4.1

SCRIPT PARA PROBAR FUNCIÓN

```
CREATE OR REPLACE FUNCTION FN_PRECIO_PRODUCTO (
    p_codigo IN VARCHAR2
) RETURN NUMBER IS
    v_precio PRODUCTO.precio_unitario%TYPE;
```

```
BEGIN
    SELECT precio_unitario
    INTO   v_precio
    FROM   PRODUCTO
    WHERE  codigo = p_codigo;

    RETURN v_precio;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('FN_PRECIO_PRODUCTO: no existe un
producto con codigo ' ||
                             NVL(p_codigo, 'NULL'));
        RETURN -1;
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('FN_PRECIO_PRODUCTO: hay mas de un
producto con codigo ' ||
                             p_codigo);
        RETURN -2;
END FN_PRECIO_PRODUCTO;
/

-- Pruebas:
BEGIN
    DBMS_OUTPUT.PUT_LINE('HM-0001 -> ' ||
FN_PRECIO_PRODUCTO('HM-0001'));   -- 45000
    DBMS_OUTPUT.PUT_LINE('XX-9999 -> ' ||
FN_PRECIO_PRODUCTO('XX-9999'));   -- -1
    DBMS_OUTPUT.PUT_LINE('hm-0001 -> ' ||
FN_PRECIO_PRODUCTO('hm-0001'));   -- -1 (distingue mayusculas)
END;
/
```

```
Function FN_PRECIO_PRODUCTO compilado

HM-0001 -> 45000
FN_PRECIO_PRODUCTO: no existe un producto con codigo XX-9999
XX-9999 -> -1
FN_PRECIO_PRODUCTO: no existe un producto con codigo hm-0001
hm-0001 -> -1

Procedimiento PL/SQL terminado correctamente.
```

4.2

SCRIPT PARA PROBAR BULK COLLECT Y FORALL

```
CREATE OR REPLACE PROCEDURE SP_VALIDAR_DESPACHO (
   p_id_producto IN NUMBER,
   p_id_bodega   IN NUMBER,
   p_cantidad    IN NUMBER
) IS
   e_stock_insuficiente EXCEPTION;
   e_cantidad_invalida  EXCEPTION;
   v_disponible EXISTENCIA.cantidad%TYPE;
BEGIN
   IF p_cantidad IS NULL OR p_cantidad <= 0 THEN
      RAISE e_cantidad_invalida;
   END IF;

   BEGIN
      SELECT cantidad
      INTO   v_disponible
      FROM   EXISTENCIA
      WHERE  id_producto = p_id_producto
      AND    id_bodega   = p_id_bodega;
   EXCEPTION
      WHEN NO_DATA_FOUND THEN
         v_disponible := 0;   -- sin fila en esa bodega: no hay unidades
   END;

   IF p_cantidad > v_disponible THEN
      RAISE e_stock_insuficiente;
   END IF;

   DBMS_OUTPUT.PUT_LINE('DESPACHO VALIDO: producto ' || p_id_producto ||
                        ', bodega ' || p_id_bodega || ', cantidad ' || p_cantidad ||
                        ' (disponible ' || v_disponible || ')');
EXCEPTION
   WHEN e_cantidad_invalida THEN
      DBMS_OUTPUT.PUT_LINE('DESPACHO RECHAZADO: la cantidad debe ser mayor que cero.');
   WHEN e_stock_insuficiente THEN
      DBMS_OUTPUT.PUT_LINE('DESPACHO RECHAZADO: producto ' || p_id_producto ||
                           ', bodega ' || p_id_bodega || ': solicitado ' ||
                           p_cantidad || ', disponible ' || v_disponible);
END SP_VALIDAR_DESPACHO;
/

-- Pruebas:
BEGIN
```

```
   SP_VALIDAR_DESPACHO(1001, 1, 10);   -- valido (hay 120)
   SP_VALIDAR_DESPACHO(1005, 1, 50);   -- rechazado (hay 4)
   SP_VALIDAR_DESPACHO(1001, 3, 5);    -- rechazado (no hay fila en bodega 3)
   SP_VALIDAR_DESPACHO(1001, 1, 0);    -- rechazado (cantidad invalida)
END;
/
```

```
Procedure SP_VALIDAR_DESPACHO compilado

DESPACHO VALIDO: producto 1001, bodega 1, cantidad 10 (disponible 135)
DESPACHO RECHAZADO: producto 1005, bodega 1: solicitado 50, disponible 4
DESPACHO RECHAZADO: producto 1001, bodega 3: solicitado 5, disponible 0
DESPACHO RECHAZADO: la cantidad debe ser mayor que cero.

Procedimiento PL/SQL terminado correctamente.
```

4.3

SCRIPT PARA PROBAR BULK COLLECT Y FORALL

```
CREATE OR REPLACE PROCEDURE SP_ANULAR_PEDIDO (
   p_id_pedido IN NUMBER
) IS
   v_estado PEDIDO.estado%TYPE;
   v_fecha  PEDIDO.fecha_pedido%TYPE;
BEGIN
   BEGIN
      SELECT estado, fecha_pedido
      INTO   v_estado, v_fecha
      FROM   PEDIDO
      WHERE  id_pedido = p_id_pedido
      FOR UPDATE;
   EXCEPTION
      WHEN NO_DATA_FOUND THEN
         RAISE_APPLICATION_ERROR(-20010, 'El pedido ' || p_id_pedido || ' no existe');
   END;

   IF v_estado = 'ANULADO' THEN
      RAISE_APPLICATION_ERROR(-20011, 'El pedido ' || p_id_pedido || ' ya esta anulado');
   END IF;

   IF SYSDATE - v_fecha > 30 THEN
      RAISE_APPLICATION_ERROR(-20012, 'El pedido ' || p_id_pedido ||
                              ' tiene mas de 30 dias y no puede anularse');
```

```
    END IF;

    UPDATE PEDIDO
    SET    estado = 'ANULADO'
    WHERE  id_pedido = p_id_pedido;

    DBMS_OUTPUT.PUT_LINE('Pedido ' || p_id_pedido || ' anulado.');
END SP_ANULAR_PEDIDO;
/

-- Pruebas (se usa un pedido de hoy creado en el bloque y se hace ROLLBACK al final,
-- para no alterar el pedido 6 que usa la Parte 6):
DECLARE
    v_id PEDIDO.id_pedido%TYPE;

    PROCEDURE probar (p_id NUMBER, p_etiqueta VARCHAR2) IS
    BEGIN
        SP_ANULAR_PEDIDO(p_id);
        DBMS_OUTPUT.PUT_LINE(p_etiqueta || ' -> OK');
    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE(p_etiqueta || ' -> ' || SQLCODE || ' ' || SQLERRM);
    END probar;
BEGIN
    probar(999, 'Pedido inexistente (999)');          -- -20010
    probar(7,   'Pedido ya anulado (7)');             -- -20011
    probar(1,   'Pedido con mas de 30 dias (1)');     -- -20012

    INSERT INTO PEDIDO (id_pedido, id_cliente, fecha_pedido, estado, total)
    VALUES (SEQ_PEDIDO.NEXTVAL, 503, SYSDATE, 'BORRADOR', 0)
    RETURNING id_pedido INTO v_id;

    probar(v_id, 'Pedido nuevo de hoy (' || v_id || ')');     -- OK
    probar(v_id, 'Mismo pedido, segunda anulacion');          -- -20011

    ROLLBACK;
END;
/
```

```
Procedure SP_ANULAR_PEDIDO compilado

Pedido inexistente (999) -> -20010 ORA-20010: El pedido 999 no existe
Pedido ya anulado (7) -> -20011 ORA-20011: El pedido 7 ya esta anulado
Pedido con mas de 30 dias (1) -> -20012 ORA-20012: El pedido 1 tiene mas de 30 dias y no puede anularse
Pedido 9 anulado.
Pedido nuevo de hoy (9) -> OK
Mismo pedido, segunda anulacion -> -20011 ORA-20011: El pedido 9 ya esta anulado

Procedimiento PL/SQL terminado correctamente.
```

4.4

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('4.4 Insertando un producto con la categoria ' ||
                         V_id_categoria || ' (inexistente)...');

    INSERT INTO PRODUCTO (
        id_producto, codigo, nombre, id_categoria,
        precio_unitario, costo_unitario, stock_minimo, activo
    ) VALUES (
        V_id_producto, 'PRB-4401', 'Producto de prueba 4.4', V_id_categoria,
        150000, 100000, 5, 'S'
    );

    -- Solo se llega aqui si la categoria si existia
    DBMS_OUTPUT.PUT_LINE('INSERT exitoso (no deberia ocurrir con esta categoria)');
    ROLLBACK;
EXCEPTION
    WHEN e_categoria_inexistente THEN
        DBMS_OUTPUT.PUT_LINE('ERROR CONTROLADO: la categoria ' || V_id_categoria ||
                             ' no existe en CATEGORIA; el producto no fue registrado.');
        DBMS_OUTPUT.PUT_LINE('Detalle de Oracle: ' || SQLERRM);
END;
/
---------------------------------------------------------------------- 4.5 (1 pt)
```

```
Salida de Script x
Tarea terminada en 0,103 segundos
Procedimiento PL/SQL terminado correctamente.
```

4.5

SCRIPT PARA PROBAR PROCEDIMIENTO Y REGISTRO

```
SET SERVEROUTPUT ON;
```

```
DECLARE
    V_resultado NUMBER;
BEGIN
    -- Forzar intencionalmente un error (ej. división por cero) para
probar el registro automático
    SELECT 10 / 0 INTO V_resultado FROM DUAL;
EXCEPTION
    WHEN OTHERS THEN
        -- Llamar a SP_REGISTRAR_ERROR para almacenar el código y
mensaje actual
        SP_REGISTRAR_ERROR('PRUEBA_BLOQUE_4_5', 'Error provocado
intencionalmente para generar evidencia');
        DBMS_OUTPUT.PUT_LINE('--> Error capturado y registrado con
éxito en la tabla LOG_ERRORES.');
END;
/

-- Verificación en la tabla LOG_ERRORES para comprobar que el
registro persistió correctamente:
SELECT id_log, origen, codigo_error, mensaje, detalle, usuario_bd,
fecha_log
FROM LOG_ERRORES
ORDER BY id_log DESC
FETCH FIRST 1 ROWS ONLY;
```

```
Procedure SP_REGISTRAR_ERROR compiled
```

```
--> Error capturado y registrado con éxito en la tabla LOG_ERRORES.


PL/SQL procedure successfully completed.


    ID_LOG ORIGEN                                                    CODIGO_ERROR MENSAJE
---------- ------------------------------------------------------------ ---------- ------------------------------------------------------------------------------------------------------------------------------------------------------
         1 PRUEBA_BLOQUE_4_5                                                  -1476 ORA-01476: divisor is equal to zero
```

5.1

```sql
-- Este trigger permite asignar automaticamente el identificador del cliente usando SEQ_CLIENTE.
-- Si esta nulo, el nombre se pone en mayusculas y sin espacios que sobren
-- y coloca el estado activo en 'S' cuando no se indica.

CREATE OR REPLACE TRIGGER TRG_CLIENTE_BI
BEFORE INSERT ON CLIENTE
FOR EACH ROW
BEGIN

    IF :NEW.id_cliente IS NULL THEN
        :NEW.id_cliente := SEQ_CLIENTE.NEXTVAL;
    END IF;

    :NEW.nombre := UPPER(TRIM(:NEW.nombre));
    :NEW.activo := NVL(:NEW.activo, 'S');
```

Salida de Script

Tarea terminada en 0,398 segundos

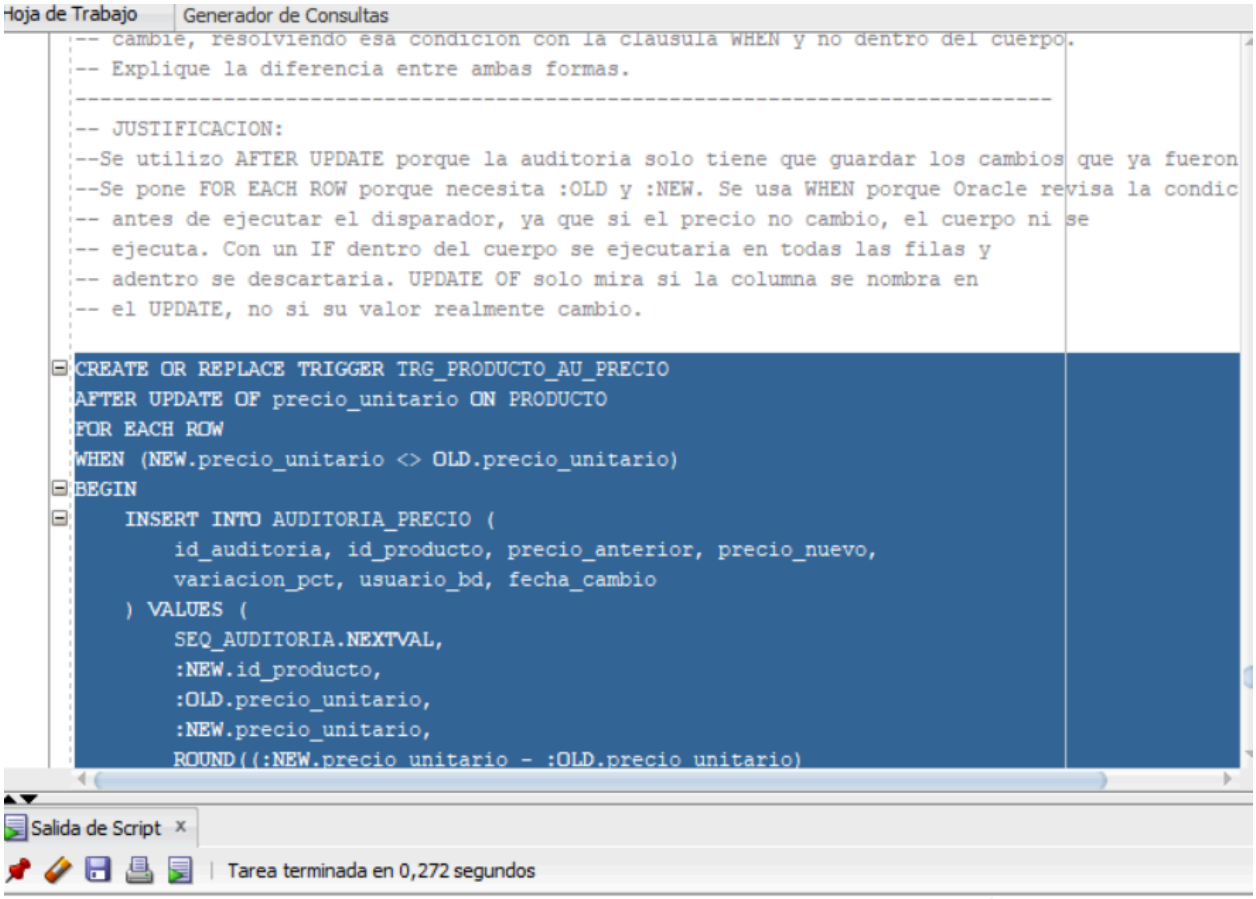
```
Trigger TRG_CLIENTE_BI compilado
```

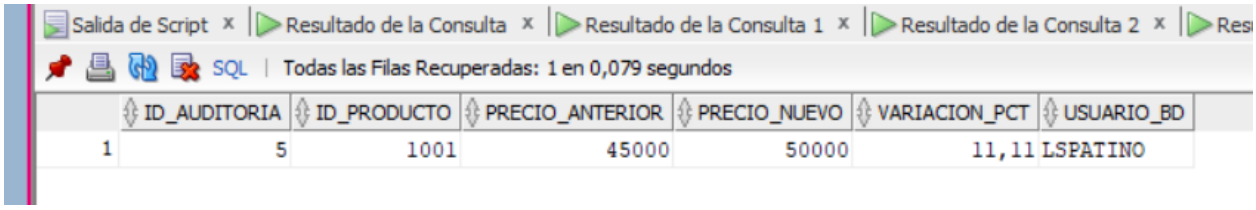
Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 1 en 0,142 segundos

	ID_CLIENTE	DOCUMENTO	NOMBRE	ACTIVO
1	500	999999999	CLIENTE PRUEBA 5.1	S

5.2





	ID_AUDITORIA	ID_PRODUCTO	PRECIO_ANTERIOR	PRECIO_NUEVO	VARIACION_PCT	USUARIO_BD
1	5	1001	45000	50000	11,11	LSPATINO

5.3

```sql
CREATE OR REPLACE TRIGGER TRG_PRODUCTO_BIU_MARGEN
BEFORE INSERT OR UPDATE ON PRODUCTO
FOR EACH ROW
DECLARE
    v_margen        CATEGORIA.margen_minimo%TYPE;
    v_precio_minimo NUMBER;
BEGIN
    BEGIN
        SELECT margen_minimo
        INTO   v_margen
        FROM   CATEGORIA
        WHERE  id_categoria = :NEW.id_categoria;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            v_margen := NULL;
    END;

    IF v_margen IS NOT NULL THEN
        v_precio_minimo := :NEW.costo_unitario * (1 + v_margen / 100);

        IF :NEW.precio_unitario < v_precio_minimo THEN
            RAISE_APPLICATION_ERROR(-20020,
                'Precio ' || :NEW.precio_unitario ||
                ' por debajo del minimo permitido ' || ROUND(v_precio_minimo, 2) ||
                ' (costo ' || :NEW.costo_unitario || ' + margen ' || v_margen || '%)');
        END IF;
    END IF;
END;
```

Salida de Script

Tarea terminada en 0,261 segundos

```
Trigger TRG_PRODUCTO_AU_PRECIO compilado


Trigger TRG_PRODUCTO_BIU_MARGEN compilado
```

Tarea terminada en 0,169 segundos

```
ERROR CONTROLADO 5.3: ORA-20020: Precio 30000 por debajo del minimo permitido 35000 (costo 28000 + margen 25%)
ORA-06512: en "LSPATINO.TRG_PRODUCTO_BIU_MARGEN", línea 19
ORA-04088: error durante la ejecución del disparador 'LSPATINO.TRG_PRODUCTO_BIU_MARGEN'
CODIGO DEL ERROR: -20020
CORRECTO: el trigger rechazo el precio por debajo del margen minimo.

Procedimiento PL/SQL terminado correctamente.
```

5.4

```
            v_cantidad := 0;
    END;

    IF v_cantidad < :NEW.cantidad THEN
        RAISE_APPLICATION_ERROR(-20030,
            'Stock insuficiente del producto ' || :NEW.id_producto ||
            ' en la bodega ' || :NEW.id_bodega ||
            ': disponible ' || v_cantidad || ', solicitado ' || :NEW.cantidad);
    END IF;

    UPDATE EXISTENCIA
    SET    cantidad    = cantidad - :NEW.cantidad
    WHERE  id_bodega   = :NEW.id_bodega
    AND    id_producto = :NEW.id_producto;

    INSERT INTO MOVIMIENTO_INVENTARIO (
        id_movimiento, id_producto, id_bodega, tipo, cantidad,
        fecha_mov, id_pedido, observacion
    ) VALUES (
        SEQ_MOVIMIENTO.NEXTVAL, :NEW.id_producto, :NEW.id_bodega, 'SALIDA',
        :NEW.cantidad, SYSDATE, :NEW.id_pedido, 'Despacho de pedido'
    );
END;
```

Salida de Script | Tarea terminada en 0,271 segundos

```
Trigger TRG_PRODUCTO_AU_PRECIO compilado


Trigger TRG_PRODUCTO_BIU_MARGEN compilado


Trigger TRG_DETALLE_AI_STOCK compilado
```

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 1 en 0,279 segundos

	ID_MOVIMIENTO	ID_PRODUCTO	ID_BODEGA	TIPO	CANTIDAD	ID_PEDIDO	OBSERVACION
1	23	1001	1	SALIDA	2	6	Despacho de pedido

5.5

```
            FROM   DETALLE_PEDIDO
            WHERE  id_pedido = v_idx;

            UPDATE PEDIDO
            SET    total = v_total
            WHERE  id_pedido = v_idx;

            v_idx := v_afectados.NEXT(v_idx);
        END LOOP;
    END AFTER STATEMENT;
```

Salida de Script | Tarea terminada en 0,778 segundos

```
Trigger TRG_DETALLE_AI_STOCK compilado

Trigger TRG_DETALLE_TOTAL compilado
```

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 1 en 0,072 segu

	ID_PEDIDO	ESTADO	TOTAL
1	6	BORRADOR	100000

5.6

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 1 en 5,158 segundos

	ID_PRODUCTO	CODIGO	PRECIO_UNITARIO	STOCK_MINIMO	STOCK_TOTAL
1	1001	HM-0001	50000	25	175

```
CREATE OR REPLACE TRIGGER TRG_V_STOCK_IO
INSTEAD OF UPDATE ON V_STOCK_CONSOLIDADO
FOR EACH ROW
BEGIN
    UPDATE PRODUCTO
    SET    precio_unitario = :NEW.precio_unitario,
           stock_minimo    = :NEW.stock_minimo
    WHERE  id_producto     = :OLD.id_producto;
END;
/


--=============================================================================
-- PARTE 6 — CASO INTEGRADOR                                    (5 puntos / 4
--=============================================================================


-------------------------------------------------------------------------------
-- PKG_VENTAS
```

Salida de Script

Tarea terminada en 0,787 segundos

```
Trigger TRG_DETALLE_TOTAL compilado


Trigger TRG_V_STOCK_IO compilado
```

```
            );
            ROLLBACK TO SAVEPOINT sp_facturar;

            SP_REGISTRAR_ERROR(
                'PKG_VENTAS.SP_FACTURAR_PEDIDO',
                v_detalle
            );

            RAISE;

    END SP_FACTURAR_PEDIDO;

END PKG_VENTAS;
/
```

Salida de Script

Tarea terminada en 0,696 segundos

```
Package PKG_VENTAS compilado

Sin errores.

Package Body PKG_VENTAS compilado
```

```
TOTAL FACTURADO: 90000
DESCUENTO APLICADO: 0
SALDO ANTES: 1250000
SALDO DESPUES: 1340000

Procedimiento PL/SQL terminado correctamente.
```

```sql
--ESTADO DEL PEDIDO DESPUES DE FACTURAR ---

SELECT id_pedido,
       id_cliente,
       estado,
       total
FROM PEDIDO
WHERE id_pedido = 6;
```

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 1 en 0,075 segundos

	ID_PEDIDO	ID_CLIENTE	ESTADO	TOTAL
1	6	501	CONFIRMADO	90000

```sql
---SALDO DEL CLIENTE DESPUES DE FACTURAR ---

SELECT id_cliente,
       cupo_credito,
       saldo_pendiente,
       FN_CUPO_DISPONIBLE(id_cliente) AS cupo_disponible
FROM CLIENTE
WHERE id_cliente = 501;
```

ida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 1 en 0,073 segundos

	ID_CLIENTE	CUPO_CREDITO	SALDO_PENDIENTE	CUPO_DISPONIBLE
1	501	8000000	1340000	6660000